Wishbone Bus Protocol Specification

Wishbone Open Core Bus Architecture

Revision 1.0
April 22, 2026

Contents

1 Introduction **5**
1.1 Overview . . . 5
1.2 Purpose and Scope . . . 5
1.3 Design Goals . . . 5

2 System Architecture **5**
2.1 Bus Topology . . . 5
2.2 Data Width and Address Space . . . 6

3 Signal Definitions **6**
3.1 Global Signals . . . 6
3.2 Master Interface Signals . . . 7
3.3 Slave Interface Signals . . . 8
3.4 Signal Descriptions . . . 8
3.4.1 CYC (Cycle) . . . 8
3.4.2 STB (Strobe) . . . 8
3.4.3 WE (Write Enable) . . . 9
3.4.4 ADR (Address Bus) . . . 9
3.4.5 DAT_O (Data Output from Master) . . . 9
3.4.6 DAT_I (Data Input to Master) . . . 9
3.4.7 ACK (Acknowledge) . . . 10

4 Transaction Protocol **10**
4.1 Transaction Types . . . 10
4.1.1 Write Transaction . . . 10
4.1.2 Read Transaction . . . 11
4.2 Transaction Timing . . . 11
4.3 Protocol Rules . . . 12
4.3.1 For Masters . . . 12
4.3.2 For Slaves . . . 12

5 Timing Diagrams **13**
5.1 Write Transaction Timing . . . 13
5.2 Read Transaction Timing . . . 14

6 Implementation Specifications **14**
6.1 Master Implementation . . . 14
6.1.1 State Machine . . . 14
6.1.2 Register Operations . . . 15
6.1.3 Operation Summary . . . 15
6.2 Slave Implementation . . . 16
6.2.1 Memory Architecture . . . 16
6.2.2 State Machine . . . 16
6.2.3 Register Operations . . . 16
6.2.4 Operation Summary . . . 17

7 System Operation **17**
7.1 Reset Sequence . . . 17
7.2 Normal Operation Flow . . . 17
7.3 Error Handling . . . 17
7.3.1 Invalid Addresses . . . 17
7.3.2 Bus Timeout . . . 18
7.3.3 Multiple Masters . . . 18

8 Advanced Features **18**
8.1 Extensibility . . . 18
8.2 Performance Optimization . . . 18
8.2.1 Burst Mode . . . 18
8.2.2 Pipelined Transfers . . . 19

9 Verification and Testing **19**
9.1 Test Cases . . . 19
9.1.1 Write Verification . . . 19
9.1.2 Read Verification . . . 19
9.1.3 Sequential Operations . . . 19
9.1.4 Edge Cases . . . 19
9.2 Functional Coverage . . . 20

10 Pin Configuration and Connectivity **20**
10.1 Master Port Connections . . . 20
10.2 Interface Matching . . . 20

11 Application Examples **20**
11.1 Simple Memory System . . . 20
11.2 Multi-Slave System . . . 21
11.3 Multi-Master System . . . 21

12 Design Guidelines **21**
12.1 Clock Domain Recommendations . . . 21
12.2 Signal Integrity . . . 21
12.3 Power Considerations . . . 21

13 Electrical Specifications **22**
13.1 DC Characteristics . . . 22
13.2 AC Characteristics . . . 22

14 Revision History **22**

15 References and Acknowledgments **22**
15.1 External Resources . . . 22
15.2 Contributors . . . 22

A Appendix A: Verilog Module Definitions **23**
A.1 Master Module . . . 23
A.2 Slave Module . . . 24

B Appendix B: Test Bench Structure **25**
B.1 Recommended Test Bench Organization 25

C Appendix C: Address Decoding Examples **25**
C.1 Multi-Slave Address Decoding 25

D Appendix D: Glossary **26**

1 Introduction

1.1 Overview

The Wishbone Bus Protocol is a simple, yet flexible open-source bus architecture designed for System-on-Chip (SoC) integration. It provides a standardized interface for connecting master and slave peripherals within a digital system. This specification defines a detailed architecture for an 8-bit Wishbone bus implementation with synchronous operation.

1.2 Purpose and Scope

This document specifies the complete behavior, signal definitions, and operational protocols for a Wishbone-based system. The specification covers:

- Signal definitions and timing characteristics
- Master and slave interface specifications
- Transaction protocols and handshaking mechanisms
- Memory access operations (read and write)
- Timing diagrams and signal relationships
- Implementation guidelines for both masters and slaves

1.3 Design Goals

1. **Simplicity**: Minimal signal count and straightforward handshaking
2. **Flexibility**: Support for multiple masters and slaves on a single bus
3. **Scalability**: Extensible to larger data widths and address spaces
4. **Synchronous Operation**: Clock-based timing for predictable behavior
5. **Ease of Implementation**: Simple state machines for master and slave logic

2 System Architecture

2.1 Bus Topology

The Wishbone bus uses a shared bus architecture where:

- Multiple master devices can initiate transactions
- Multiple slave devices can respond to master requests
- A central arbiter (not specified in this document) arbitrates master access
- All signals are synchronized to a common clock signal

2.2 Data Width and Address Space

Parameter	Value
Data Width (DAT)	8 bits
Address Width (ADR)	8 bits
Maximum Addressable Memory	256 locations (0x00 to 0xFF)
Clock Frequency	Application Dependent

Table 1: System Parameters

3 Signal Definitions

3.1 Global Signals

Signal Name	Width	Description
CLK	1 bit	Clock signal. All state changes occur on rising edges.
RST	1 bit	Active-high asynchronous reset. Resets all modules to initial state.

Table 2: Global Signals

3.2 Master Interface Signals

Signal	Width	Dir.	Description
CYC	1 bit	Out	Cycle signal. Asserted by master to indicate an active bus cycle.
STB	1 bit	Out	Strobe signal. Asserted to indicate valid address and data.
WE	1 bit	Out	Write Enable. High for write operations, low for read operations.
ADR	8 bits	Out	Address bus. Specifies the target memory address.
DAT_O	8 bits	Out	Data Output. Write data from master to slave.
ACK	1 bit	In	Acknowledge signal. Asserted by slave to confirm transaction completion.
DAT_I	8 bits	In	Data Input. Read data from slave to master.
BUSY	1 bit	Out	Busy flag. Indicates master is in a transaction.
START	1 bit	In	Start signal. Request from external source to initiate a transaction.
ADR_IN	8 bits	In	Input address for the transaction.
DAT_IN	8 bits	In	Input write data for the transaction.

Table 3: Master Interface Signals

3.3 Slave Interface Signals

Signal	Width	Dir.	Description
CYC	1 bit	In	Cycle signal from master.
STB	1 bit	In	Strobe signal from master.
WE	1 bit	In	Write Enable signal from master.
ADR	8 bits	In	Address bus from master.
DAT_O	8 bits	In	Data from master (write data).
ACK	1 bit	Out	Acknowledge signal to master.
DAT_I	8 bits	Out	Data to master (read data).

Table 4: Slave Interface Signals

3.4 Signal Descriptions

3.4.1 CYC (Cycle)

- **Direction**: Master Output / Slave Input
- **Width**: 1 bit
- **Timing**: Asserted when a bus cycle begins, held throughout the cycle, deasserted when the cycle ends
- **Purpose**: Indicates to all slaves that a valid bus cycle is in progress
- **Protocol**: The master asserts CYC at the start of a transaction and keeps it asserted until the slave acknowledges with ACK

3.4.2 STB (Strobe)

- **Direction**: Master Output / Slave Input
- **Width**: 1 bit
- **Timing**: Asserted synchronously with CYC
- **Purpose**: Indicates that the address (ADR) and write data (DAT_O) are valid
- **Protocol**: Only the slave(s) selected by the address should respond when both CYC and STB are high

3.4.3 WE (Write Enable)

- **Direction**: Master Output / Slave Input
- **Width**: 1 bit
- **Timing**: Valid during CYC and STB
- **Purpose**: Specifies the transaction type
- **Values**:
  - WE = 1: Write operation (master to slave data transfer)
  - WE = 0: Read operation (slave to master data transfer)

3.4.4 ADR (Address Bus)

- **Direction**: Master Output / Slave Input
- **Width**: 8 bits
- **Timing**: Valid when CYC and STB are high
- **Purpose**: Specifies the target memory location or register address
- **Range**: 0x00 to 0xFF (256 addressable locations)

3.4.5 DAT_O (Data Output from Master)

- **Direction**: Master Output / Slave Input
- **Width**: 8 bits
- **Timing**: Valid when CYC, STB, and WE are high
- **Purpose**: Carries write data from master to slave
- **Hold Time**: Data is held stable until ACK is asserted

3.4.6 DAT_I (Data Input to Master)

- **Direction**: Slave Output / Master Input
- **Width**: 8 bits
- **Timing**: Valid when CYC and STB are high, and WE is low (read operation)
- **Purpose**: Carries read data from slave to master
- **Behavior**: Slave must drive this bus with the requested data at the target address

3.4.7 ACK (Acknowledge)

- **Direction**: Slave Output / Master Input
- **Width**: 1 bit
- **Timing**: Asserted by slave in response to a valid transaction
- **Purpose**: Signals transaction completion to the master
- **Protocol**:
  - Asserted when CYC and STB conditions are met
  - Held high for one clock cycle
  - Signals both read and write completion

4 Transaction Protocol

4.1 Transaction Types

4.1.1 Write Transaction

A write transaction transfers data from the master to the slave at a specified address.

Sequence:

1. Master asserts START signal with ADR_IN (target address) and DAT_IN (write data)
2. On next clock edge, master sets:
   - CYC = 1
   - STB = 1
   - WE = 1 (write mode)
   - ADR = ADR_IN
   - DAT_O = DAT_IN
   - BUSY = 1
3. Slave observes CYC=1, STB=1, WE=1 and valid ADR
4. Slave captures DAT_O and writes to memory at address ADR
5. Slave asserts ACK = 1
6. Master observes ACK = 1 on next clock edge
7. Master deasserts:
   - CYC = 0
   - STB = 0
   - BUSY = 0
8. Slave deasserts ACK = 0
9. Transaction completes

4.1.2 Read Transaction

A read transaction transfers data from the slave to the master from a specified address.

Sequence:

1. Master asserts START signal with ADR_IN (target address), DAT_IN = 0
2. On next clock edge, master sets:
   - CYC = 1
   - STB = 1
   - WE = 0 (read mode)
   - ADR = ADR_IN
   - BUSY = 1
3. Slave observes CYC=1, STB=1, WE=0 and valid ADR
4. Slave reads memory at address ADR and drives DAT_I with the value
5. Slave asserts ACK = 1
6. Master observes ACK = 1 and captures DAT_I on the same clock edge
7. Master deasserts:
   - CYC = 0
   - STB = 0
   - BUSY = 0
8. Slave deasserts ACK = 0
9. Transaction completes

4.2 Transaction Timing

Key Timing Characteristics:

- **Single-Cycle Transactions**: Both read and write complete in exactly 2 clock cycles from START assertion
- **Setup Time**: Address and data must be stable before clock edge
- **Hold Time**: Address and data are held through ACK assertion
- **Clock-to-ACK Delay**: Slave ACK appears synchronously on clock edge following valid strobe signals

4.3 Protocol Rules

4.3.1 For Masters

1. Master must maintain CYC and STB asserted until ACK is received
2. Address (ADR) and write data (DAT_O) must be stable while CYC and STB are asserted
3. Master must deassert CYC and STB within one clock cycle of observing ACK
4. BUSY signal must accurately reflect transaction state
5. START signal should be asserted for one clock cycle only

4.3.2 For Slaves

1. Slave must respond with ACK when both CYC and STB are high and addressing matches
2. For write transactions, slave must capture DAT_O and store at address ADR
3. For read transactions, slave must immediately drive DAT_I with data from address ADR
4. Slave must deassert ACK after one clock cycle
5. Slave must not respond to transactions when CYC or STB are low

5 Timing Diagrams

5.1 Write Transaction Timing

```
Clock      ___     ___     ___     ___     ___     ___     ___
           ___|   |___|   |___|   |___|   |___|   |___|   |___|
START      ___
           ____|_______________________

CYC        ___
           ____|_________________|___________

STB        ___
           ____|_________________|___________

WE         ___
           ____________________________________

ADR        ___     [Valid Address]     [Valid Address]
           ____|_________________|___________

DAT_O      ___     [Write Data]        [Write Data]
           ____|_________________|___________

ACK        ___
           ______________________|___________

BUSY       ___
           ____|_________________|___________

           Cycle 0  1      2      3      4
```

Figure 1: Write Transaction Timing Diagram

5.2 Read Transaction Timing

```
Clock       ___     ___     ___     ___     ___     ___     ___
            ___|   |___|   |___|   |___|   |___|   |___|   |___|
START       ___
            ____|_______________________

CYC         ___
            ____|_________________|___________

STB         ___
            ____|_________________|___________

WE          ___
            ____________________________________

ADR         ___     [Valid Address]     [Valid Address]
            ____|_________________|___________

DAT_I       ___     [Read Data]         [Read Data]
            ____|_________________|___________

ACK         ___
            ______________________|___________

BUSY        ___
            ____|_________________|___________

            Cycle 0  1      2      3      4
```

Figure 2: Read Transaction Timing Diagram

6 Implementation Specifications

6.1 Master Implementation

6.1.1 State Machine

The master employs a simple two-state state machine:

IDLE State:

- CYC = 0, STB = 0, BUSY = 0
- Waits for START signal assertion
- On START: transition to BUSY state

BUSY State:

- CYC = 1, STB = 1, BUSY = 1
- Holds transaction signals until ACK received
- On ACK: deassert signals and return to IDLE state

6.1.2 Register Operations

Listing 1: Master State Machine Logic

```
always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        cyc <= 0;
        stb <= 0;
        we <= 0;
        adr <= 0;
        dat_o <= 0;
        busy <= 0;
    end
    else
    begin
        // Transition from IDLE to BUSY
        if (!busy && start)
        begin
            cyc <= 1;
            stb <= 1;
            we <= 1;
            adr <= adr_in;
            dat_o <= dat_in;
            busy <= 1;
        end
        // Transition from BUSY to IDLE
        else if (busy && ack)
        begin
            cyc <= 0;
            stb <= 0;
            busy <= 0;
        end
    end
end
```

6.1.3 Operation Summary

- All state changes occur on clock rising edges
- Reset is asynchronous and active-high
- Master initiates transactions via START signal
- BUSY flag prevents overlapping transactions

- Data width is 8 bits for simplicity

6.2 Slave Implementation

6.2.1 Memory Architecture

The slave contains an internal 256-byte addressable memory:

Listing 2: Slave Memory Declaration

```
reg [7:0] memory [0:255];
```

6.2.2 State Machine

The slave employs a request-response protocol:

IDLE State:

- ACK = 0
- Monitors CYC and STB signals
- On CYC=1 and STB=1: process transaction

RESPOND State:

- ACK = 1 for one clock cycle
- For write: captures DAT_O into memory[ADR]
- For read: drives DAT_I with memory[ADR]

6.2.3 Register Operations

Listing 3: Slave State Machine Logic

```
always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        ack <= 0;
    end
    else
    begin
        if (cyc && stb)
        begin
            ack <= 1;

            if (we)
                memory[adr] <= dat_o;    // Write operation
            else
                dat_i <= memory[adr];    // Read operation
        end
        else
            ack <= 0;
```

```
    end
end
```

6.2.4 Operation Summary

- All transactions complete in one clock cycle
- Write operations are synchronous (data captured on clock edge)
- Read operations provide data combinatorially through DAT_I
- Slave supports full 8-bit address range (0x00 to 0xFF)
- Slave supports all 256 memory locations

7 System Operation

7.1 Reset Sequence

Upon assertion of RST:

1. All flip-flops in master are reset to 0
2. All flip-flops in slave are reset to 0
3. Bus signals (CYC, STB, WE, ACK) are deasserted
4. Both master and slave return to IDLE state
5. Memory contents in slave are preserved (not reset)

7.2 Normal Operation Flow

1. System starts with RST = 1 for at least one clock cycle
2. RST is deasserted; both master and slave enter normal operation
3. External stimulus provides START, ADR_IN, and DAT_IN to master
4. Master begins transaction as per protocol
5. Slave responds with data or acknowledgment
6. Transaction completes with CYC and STB deasserted
7. System ready for next transaction

7.3 Error Handling

7.3.1 Invalid Addresses

In this implementation, all 256 addresses (0x00-0xFF) are valid and accessible. Out-of-range addresses wrap around due to 8-bit address bus.

7.3.2 Bus Timeout

This specification does not include bus timeout mechanisms. Masters must ensure:

- Slave responds with ACK within finite time
- No deadlock conditions exist
- All addressed locations are valid

7.3.3 Multiple Masters

If multiple masters are present in the system:

- An external arbiter must control which master drives the bus
- Only one master can assert CYC at a time
- Arbitration logic is outside this specification

8 Advanced Features

8.1 Extensibility

This specification can be extended to support:

- **Wider Data Paths**: Increase DAT_O and DAT_I widths to 16, 32, or 64 bits
- **Larger Address Space**: Extend ADR width beyond 8 bits for larger memory
- **Burst Transactions**: Extend protocol to support multiple data transfers per cycle initiation
- **Byte Enables**: Add SEL (select) signals for sub-word access
- **Error Signaling**: Add ERR signal for fault conditions
- **Ready Signals**: Add RDY signal for pipelined transactions

8.2 Performance Optimization

8.2.1 Burst Mode

Extend the protocol to support multiple consecutive accesses without releasing the bus:

- Keep CYC asserted across multiple transactions
- Perform address and data changes between transactions
- Only release bus when all data transfers complete

8.2.2 Pipelined Transfers

Allow new transactions to begin before previous ones complete:

- Requires additional buffering
- Improves bus utilization
- Requires more complex arbiter logic

9 Verification and Testing

9.1 Test Cases

9.1.1 Write Verification

1. Write 0xA5 to address 0x10
2. Verify memory location 0x10 contains 0xA5
3. Repeat for addresses 0x00, 0x80, 0xFF

9.1.2 Read Verification

1. Pre-load memory with known patterns
2. Read from various addresses
3. Verify returned data matches memory contents
4. Test all addresses in range

9.1.3 Sequential Operations

1. Perform multiple read-write sequences
2. Verify BUSY signal prevents overlapping operations
3. Verify no data corruption

9.1.4 Edge Cases

1. Test address wraparound (0xFF to 0x00)
2. Test boundary addresses (0x00, 0xFF)
3. Test all possible 8-bit data values
4. Test reset during operation

9.2 Functional Coverage

All operations must achieve functional coverage for:

- **Read Operations**: All 256 addresses read with valid data returned
- **Write Operations**: All 256 addresses written with data correctly stored
- **Reset Behavior**: Signals correctly initialized on RST
- **State Transitions**: All master state transitions exercised
- **ACK Timing**: ACK timing verified across multiple transactions

10 Pin Configuration and Connectivity

10.1 Master Port Connections

Signal	Master	Slave
CLK	Input	Input
RST	Input	Input
CYC	Output	Input
STB	Output	Input
WE	Output	Input
ADR[7:0]	Output	Input
DAT_O[7:0]	Output	Input
DAT_I[7:0]	Input	Output
ACK	Input	Output

Table 5: Master-Slave Signal Connectivity

10.2 Interface Matching

- All outputs from one module connect to inputs of the other
- Bus signals (CYC, STB, WE, ADR, DAT_O) are point-to-point from master to slave
- Response signals (ACK, DAT_I) are point-to-point from slave to master
- Clock and reset are distributed to all modules

11 Application Examples

11.1 Simple Memory System

A basic system with a single master and single slave:

- Master: Microcontroller or state machine
- Slave: 256-byte RAM
- Application: Memory-mapped I/O and data storage

11.2 Multi-Slave System

Extension to multiple slave devices:

- Single master drives up to 256 addressable slaves
- Each slave responds to its assigned address range
- Slave selection based on upper address bits

11.3 Multi-Master System

Multiple masters sharing a single bus:

- Central arbiter grants bus access to masters
- Prevents address conflicts through arbitration
- Requires priority or round-robin arbitration

12 Design Guidelines

12.1 Clock Domain Recommendations

- **Synchronous Design**: All Wishbone signals should operate in the same clock domain
- **Clock Frequency**: Determined by slowest component in the system
- **Reset Synchronization**: RST should be synchronized to CLK before distribution
- **Clock Distribution**: Use a dedicated clock tree to minimize skew

12.2 Signal Integrity

- **Line Termination**: Implement appropriate termination for high-speed designs
- **Ground Reference**: Maintain clean ground planes for signal integrity
- **Impedance Control**: Control trace impedance to minimize reflections
- **Crosstalk**: Route address and data buses away from high-speed signals

12.3 Power Considerations

- **Power Supplies**: Separate supply rails for analog and digital circuits when necessary
- **Decoupling**: Place bypass capacitors near all power pins
- **Ground Distribution**: Use multiple vias for low impedance ground connections

13 Electrical Specifications

13.1 DC Characteristics

Parameter	Min	Max
Supply Voltage (VDD)	3.0 V	3.6 V
Logic High (VIH)	2.0 V	VDD
Logic Low (VIL)	0 V	0.8 V
Output High (VOH)	VDD-0.2 V	VDD
Output Low (VOL)	0 V	0.2 V

Table 6: DC Electrical Specifications (Typical for 3.3V CMOS)

13.2 AC Characteristics

Parameter	Value
Setup Time (tSU)	2 ns (typical)
Hold Time (tH)	1 ns (typical)
Propagation Delay (tPD)	3 ns (typical)
Clock-to-Output (tCO)	5 ns (typical)

Table 7: AC Timing Characteristics

14 Revision History

Revision	Date	Description
1.0	2026-04-22	Initial specification document

Table 8: Document Revision History

15 References and Acknowledgments

15.1 External Resources

- Wishbone Architecture Documentation
- AMBA Bus Protocol Specification
- IEEE Standard 1735 (Intellectual Property Protection)

15.2 Contributors

- Wishbone Open Core Community
- Protocol Implementation Team

A Appendix A: Verilog Module Definitions

A.1 Master Module

Listing 4: Complete Wishbone Master Module

```
module wishbone_master(
    input clk,
    input rst,
    input ack,
    input [7:0] dat_i,
    input start,
    input [7:0] adr_in,
    input [7:0] dat_in,

    output reg cyc,
    output reg stb,
    output reg we,
    output reg [7:0] dat_o,
    output reg [7:0] adr,
    output reg busy
);

always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        cyc <= 0;
        stb <= 0;
        we <= 0;
        adr <= 0;
        dat_o <= 0;
        busy <= 0;
    end
    else
    begin
        if (!busy && start)
        begin
            cyc <= 1;
            stb <= 1;
            we <= 1;
            adr <= adr_in;
            dat_o <= dat_in;
            busy <= 1;
        end
        else if (busy && ack)
        begin
            cyc <= 0;
            stb <= 0;
            busy <= 0;
        end
    end
```

```
end

endmodule
```

A.2 Slave Module

Listing 5: Complete Wishbone Slave Module

```
module wishbone_slave(
    input clk,
    input rst,
    input cyc,
    input stb,
    input we,
    input [7:0] adr,
    input [7:0] dat_o,

    output reg ack,
    output reg [7:0] dat_i
);

reg [7:0] memory [0:255];

always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        ack <= 0;
    end
    else
    begin
        if (cyc && stb)
        begin
            ack <= 1;

            if (we)
                memory[adr] <= dat_o;
            else
                dat_i <= memory[adr];
        end
        else
            ack <= 0;
    end
end

endmodule
```

B Appendix B: Test Bench Structure

B.1 Recommended Test Bench Organization

Listing 6: Testbench Framework

```
module wishbone_tb;

reg clk, rst;
reg [7:0] adr_in, dat_in;
reg start;

wire cyc, stb, we, ack, busy;
wire [7:0] adr, dat_o, dat_i;

wishbone_master master(
    .clk(clk), .rst(rst), .start(start),
    .adr_in(adr_in), .dat_in(dat_in),
    .cyc(cyc), .stb(stb), .we(we),
    .adr(adr), .dat_o(dat_o), .dat_i(dat_i),
    .ack(ack), .busy(busy)
);

wishbone_slave slave(
    .clk(clk), .rst(rst),
    .cyc(cyc), .stb(stb), .we(we),
    .adr(adr), .dat_o(dat_o),
    .ack(ack), .dat_i(dat_i)
);

initial begin
    // Test procedures
end

endmodule
```

C Appendix C: Address Decoding Examples

C.1 Multi-Slave Address Decoding

For systems with multiple slaves, use address decoding:

Listing 7: Address Decoder Example

```
// Slave 0: addresses 0x00-0x3F
// Slave 1: addresses 0x40-0x7F
// Slave 2: addresses 0x80-0xBF
// Slave 3: addresses 0xC0-0xFF

wire slave0_sel = (adr[7:6] == 2'b00);
wire slave1_sel = (adr[7:6] == 2'b01);
```

```
wire slave2_sel = (adr[7:6] == 2'b10);
wire slave3_sel = (adr[7:6] == 2'b11);
```

D Appendix D: Glossary

Term	Definition
ACK	Acknowledge signal from slave indicating transaction completion
ADR	Address bus carrying memory or register location
Bus Cycle	Complete transaction from CYC assertion to ACK
CYC	Cycle signal indicating active bus transaction
DAT	Data bus (DAT_O for output, DAT_I for input)
Master	Initiator of bus transactions
Slave	Responder to bus transactions
STB	Strobe signal indicating valid address and data
WE	Write Enable signal (1=write, 0=read)

Table 9: Glossary of Terms